Science of Computer Programming 165 (2018) 24–37

Contents lists available at ScienceDirect

Science of Computer Programming

www.elsevier.com/locate/scico

Design and implementation of a cross-layer IoT protocol

Davi Resner *, Gustavo Medeiros de Araujo, Antônio Augusto Fröhlich

Software/Hardware Integration Lab, Federal University of Santa Catarina, PO Box 476, 88040-900, Florianópolis, SC, Brazil

ARTICLE INFO

Article history:
Received 1 February 2017
Received in revised form 14 August 2017
Accepted 17 August 2017
Available online 24 August 2017

Keywords:
Cross-layer design
Communication protocols
Internet of Things
Wireless Sensor Networks

ABSTRACT

Cross-layer communication protocols can significantly improve several aspects of wireless networks, particularly energy consumption and bandwidth. However, they break with the traditional layered architecture that has been so successful for over 30 years. A fully fledged cross-layer stack requires a sophisticated software design to be maintainable and reusable. The Trustful Space-Time Protocol (TSTP) is a cross-layer protocol designed to deliver authenticated, encrypted, timed, and georeferenced messages containing data compliant with the International System of Units (SI) in a resource-efficient way. By integrating shared data from multiple networking services into a single communication infrastructure, TSTP is able to eliminate replication of information across services, achieving small overhead regarding control messages. The complexity of TSTP's features, its broad range – from the application to the Medium Access Control, – and its experimental nature bring diverse requirements beyond those usually considered in most software designs. In this work, we show how we avoided a monolithic implementation of the cross-layer approach with a component-based design, exploring template metaprogramming techniques to adapt and combine basic building blocks. An event-driven architecture that makes use of zero-copy buffers and metadata is used to handle crosscutting concerns. We validate the proposed design with an implementation for IEEE 802.15.4 networks and a set of OMNet++ simulations for relevant scenarios, showing that we achieve a light TSTP implementation that is successfully able to save energy and bandwidth while keeping a delivery ratio close to 100%.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

Wireless Sensor Networks (WSNs) have been the focus of intense research for well over a decade by now. Several physical layers have been proposed, along with a myriad of medium access and routing protocols [11] [19]. Such protocols have been made energy-aware [17]; aggregation and fusion strategies have been employed [14]; basic infrastructures have been enriched with location [20], timing [26], and security protocols [10]; operating systems have been designed to support higher-level abstractions [3], along with large-scale management systems built to handle the produced data properly [12]. We are currently seeing these networks being connected to the Internet of Things (IoT).

Extensive research is also being carried out on cross-layer optimizations for wireless communication [8] and WSN [16] protocols. These works focus on taking into account related information given by one layer to efficiently make decisions at a different layer in the communication stack. A cross-layer protocol design can eliminate a large number of control

* Corresponding author.
E-mail addresses: davir@lisha.ufsc.br (D. Resner), gustavo@lisha.ufsc.br (G. Medeiros de Araujo), guto@lisha.ufsc.br (A.A. Fröhlich).

http://dx.doi.org/10.1016/j.scico.2017.08.008
0167-6423/© 2017 Elsevier B.V. All rights reserved.

messages and improve decision making within the protocol by pig-tailing control information on ordinary data packets and subsequently organizing and sharing that information with all protocol components.

The main advantage of the traditional layered model is the inherent ability to induce highly cohesive, loosely coupled modular software implementations since layers see each other as black-boxes with a well-defined and small interface. Conversely, a cross-layer protocol calls for close component interactions, challenging designers aiming for traditional software engineering quality metrics such as modularity, reusability, and maintainability. Furthermore, embedded IoT devices usually have low processing power and very limited energy budgets, so the network protocol stack, which is often a major part of the whole system's functionality, must be efficiently implemented.

In this work, we present the component-based design of the cross-layer *Trustful Space-Time Protocol* (TSTP) [23], highlighting the close interactions among MAC, router, location, and time synchronization components. We discuss the techniques that were used in this design on the *Embedded Parallel Operating System* (EPOS) [13] to demonstrate that the tight relationships in the cross-layered approach can be modeled and implemented avoiding a monolithic, tightly-coupled software. We use template metaprogramming techniques to implement an event-driven architecture that moves packets stored in zero-copy, metadata-enriched buffers across protocol components efficiently at the same time as it eliminates unnecessary dependencies. We present a set of simulations that corroborate the benefits of the cross-layer approach regarding energy consumption and bandwidth when compared to other non-cross-layered protocols. We also evaluate the component-based implementation concerning size and performance to demonstrate the small overhead incurred by the chosen techniques, which is also confirmed by comparisons with the same traditional protocols used in the simulations. EPOS and TSTP are written in C++ and are freely available online at https://epos.lisha.ufsc.br/. The EPOSMote hardware used to automate UFSC's Smart Solar Building, the real IoT platform used for the evaluations in this paper, is a free hardware project available at the same site.

We present the general cross-layer design of TSTP in Section 2, with a focus on the MAC and routing interaction, which encompasses much of the challenges that arise in the cross-layer design. Section 3 explains the techniques used to achieve a light and modular implementation of that design. In Section 4 we evaluate the protocol's design and implementation by using simulations and measurements on a real IoT device. Related work is presented in Section 5, and Section 6 concludes the work.

2. Cross-layer protocol design

Cross-layer designs have been shown to be very efficient in optimizing wireless networks [27]. In practice, cross-layer designs usually work by taking information from one or more layers of a typical layered stack to optimize a set of parameters or make a decision in another layer or set of layers [8]. From the myriad of cross-layer proposals, a minority involves the application layer, and even fewer encompass the complete stack to present a truly application-oriented, domain-specific solution.

The *Trustful Space-Time Protocol* (TSTP) [23] is an application-oriented, cross-layer protocol for *Cyber-Physical Systems* (CPS) on a WSN or the IoT. Instead of focusing on keeping the original protocol interfaces in a modular, layered architecture with shared data, TSTP focuses on efficiently providing functionality recurrently needed by such systems: trusted, timed, geo-referenced, SI-compliant data that is resource-efficiently delivered to a sink or gateway. TSTP grants these functionalities directly to the application in the form of a complete communication solution, which allows the design of optimized, synergistic cooperation of sub-protocols while eliminating the need for additional heterogeneous software layers that come with an integration cost and often result in replication of data.

TSTP integrates time synchronization, spatial localization, security, MAC, routing, and a data-centric API. In previous work [23], we evaluated the whole design analytically and showed that the cross-layer integration of those services yields, in some scenarios, a node deployment 31% faster than the deployment time achieved by a design with all these services operating individually, as in a traditional layered stack. Here we extend the analysis to the integration, implementation, and interaction of the components, particularly the two most impacting ones (MAC and routing), and enrich it with simulations using the Castalia [2] framework on the OMNeT++ [18] simulator.

2.1. Messages, space and time

Fig. 1 shows the format of the header present in every TSTP message. It consists mostly of information necessary to characterize the *data* being transmitted: *where* (`Origin(x,y,z)`) and *when* (`Origin(t)`) it was produced, and *until when* (`Expiry`) it should be consumed. Other fields pertain coordinates scaling, message relay, or serve to keep nodes synchronized in space and time. TSTP messages are signed and their payloads are encrypted [24], so no explicit error-detecting codes are used (corrupted messages will fail decryption).

TSTP synchronizes nodes in time with high accuracy ($\approx$1 µs for IEEE 802.15.4) using the *Speculative Precision Time Protocol* [25]. If the speculative technique fails because the network is too silent at a moment and the estimated clock skew reaches half of the maximum tolerated value, then a message with the `Time Request` bit is sent to neighbors (1 hop), which react replying with an empty message (containing only the header) just to fulfill the requirements of the time synchronization algorithm. Nodes are synchronized in space using a speculative *Heuristic Cooperative Calibration Positioning System* (HeCoPs) [20]. Each message carries the last hop coordinates (`Last Hop(x,y,z)`), that node's confidence on its

Header Format									
Bits: 3	**1**	**2**	**2**	**8**	**3*sb**	**64**	**3*sb + tb**	**64**	**0 or 32**
Message Type	Time Request	Spatial Scale	Temporal Scale	Location Confidence	Last Hop (x,y,z)	Last Hop Timestamp	Origin (x,y,z,t)	Expiry	Location Deviation

Fig. 1. TSTP message header format (the number of bits in spatial (sb) and temporal (tb) coordinates are defined by the `Spatial Scale` and `Temporal Scale` fields).

Microframe Format				
Bits: 1	**11**	**12**	**32**	**16**
All Listen	Count	Id	Distance	CRC

Fig. 2. TSTP microframe format.

location (`Location Confidence`, anchors = 100%) and the deviation (`Location Deviation`) it observed on the signal (e.g. radio or ultrasound) propagation used to estimate its location. Neighbors with high confidence on their locations are used for trilateration and the signal deviation observed by an adjacent node is used to correct the calculations. Both these protocols require anchor nodes that have other means to acquire their clocks and spatial location (e.g. GPS for time and space, predefined coordinates for a static node). For the comparisons with other protocols presented in this paper, these two protocols have been implemented at the application-level, respecting their layered organization and therefore not speculating on foreign messages.

2.2. Medium access control and routing

TSTP MAC [22] is the component responsible for interfacing the protocol with the network with a special care for energy consumption. Its design follows the general principles of RB-MAC [1] (Section 5): senders send a long preamble composed of *microframes* before each message, such that at most one message occupies the channel at every period S (for a full period); sensor nodes sleep for most of the time and, when they receive a message, nodes closer to the destination become relay candidates and use their distances to the message's destination to derive the time offset for *Clear Channel Assessment* (CCA) and transmission: the relay candidate closest to the destination assesses the channel earlier and wins the contention, resulting in a greedy, fully-reactive, geographic routing. The offset δ for a message m at any given node is locally calculated as:

$$\delta(m) = \left\lfloor \frac{|R + D - D_m|}{R} \times \frac{S}{g} \right\rfloor \times g \tag{1}$$

with D representing the current node's distance to the message's destination, D_m representing the message sender's distance to the destination, R representing a network-wide parameter corresponding to the radio range of the nodes, and g representing the time gap between a transmitter sensing an available channel and finishing transmission of enough symbols for other nodes to detect a busy channel (8 symbols, or 320 µs for IEEE 802.15.4). This equation makes nodes closer to the destination wake up earlier, normalizes the offset to the period S (since nodes only route messages that come from nodes more distant to the destination – see Algorithm 1, – it holds that $D < D_m$), and ensures that, if the destination is extremely far away, neighboring nodes (that are at the same discrete point in space in the scale of the total distance) will still have different offsets.

TSTP takes a toll on senders and on overall latency to reach ultra-low power consumption at receivers. The time between two consecutive microframes has a lower bound T_u defined by the underlying communication technology (192 µs in IEEE 802.15.4), and the time T_s to send a single microframe is fixed and known. TSTP MAC defines that the idle listening duration which happens every period S is $2T_s + T_u$. Therefore, by setting the desired idle listening duty cycle d, we can determine the period S as follows:

$$S = T_s + (N_{MF} - 1) \times (T_s + T_u) \tag{2}$$

where N_{MF} is the number of microframes to be sent before each message:

$$N_{MF} = 1 + \frac{\frac{2T_s + T_u}{d} + T_u + T_s - 1}{T_u + T_s} \tag{3}$$

An implementation of TSTP MAC over an IEEE 802.15.4 2450 MHz DSSS PHY layer uses values for the period S greater than 116 ms to reach an idle listening duty cycle smaller than 1% [22]. If energy consumption is not critical, shorter periods can be used to reduce latency as long as they are not too small, what could compromise CCA (this limit is captured as g in Equation (1) and depends on the technology of the network; $g = 320$ µs for IEEE 802.15.4).

TSTP microframes follow the format depicted in Fig. 2. The `All Listen` field requires all listening nodes to receive a message, even those that wouldn't be candidates to relay it because they are further away from the destination. It is used for control messages and for messages whose destination is not a gateway and therefore do not follow the default routing algorithm. The `Count` field is initialized by the sending node in accordance with S and decremented for each

subsequent microframe until the transmission of the message (`Count` = 0). This enables relay candidates to sleep until the time when the message is sent as long as they have listened to at least one microframe. The `Id` field is initialized at the origin (i.e. at the node that effectively produced the data) and accompany the message unmodified across the network until its destination. The purpose of this field is to reduce replication along the route, but not much effort is dedicated to making it unique since the destination has all the information it needs to detect a duplicate in the message's header (i.e. `Origin(x,y,z,t)`). It can be a hash from the origin coordinates, particularly the time, or even a random number. The `Distance` field holds the calculated distance between the transmitting node and the message's destination, and allows listening nodes to discard a message by listening to a single microframe when `All Listen = 0` and `Distance` is smaller than the listener's distance to the gateway. `CRC` is used to discard corrupted microframes.

Messages being routed are kept on each involved node i in a queue Q_i. Each entry e_Q in Q_i represents a message m that is scheduled for transmission or retransmission. In addition to the message, e_Q also holds m's *Id* (extracted from the microframes that preceded it), its *Expiry* (extracted from its header), its destination *Dst* (either a previously assigned gateway/sink or extracted from the payload for specific control messages), and a $\delta(m)$ calculated as described earlier in this section.

TSTP uses implicit acknowledgment (ACK) to confirm the routing of messages. A node *only* removes a message from its queue when another message with the same *Id* is overheard in the network (that is, when another node handles the forwarding of that message towards its destination). The only case in which an explicit ACK is used is when the message reaches its final destination: that node must retransmit the same message, just to acknowledge the last forwarder and any neighbors that might still have that message queued. Since this last ACK has a `Last Hop` coordinate identical to the destination, the calculated distance of zero won't be matched by any other node in the vicinity. It, therefore, serves purely as a final ACK. This algorithm is explained in further detail in Section 2.2.2.

2.2.1. Spatial distortion

TSTP's transmission time offset $\delta(m)$, given by Equation (1), is made sensitive to other routing metrics according to application's needs. For example, it may take into account the remaining battery charge or buffer space of a node to ensure that nodes in an optimal path are not going to have their batteries depleted too quickly [17]. To accomplish this, we use the notion that those additional metrics *distort space*. A node running out of memory or consuming too much energy can *stretch* space, increasing its distance to the destination so that other nodes in the vicinity become more likely to win the contention to retransmit a message. Conversely, a node transmitting a message that is close to expiring can *shrink* space, reducing its distance to the destination and thus increasing the chances of winning the contention and getting the message transmitted earlier. Equation (4) redefines the offset equation introducing a distortion coefficient $\alpha \in [0, 1]$:

$$\delta(m) = \left\lfloor \alpha \times \frac{|R - D_m + D|}{R} \times \frac{S}{g} \right\rfloor \times g \tag{4}$$

The distortion coefficient α defines how much other metrics influence the perceived distance, and hence the offset used for contention. It may take into account virtually any metric of interest, as long as it is a real number in the interval $[0, 1]$. A value of $\alpha < 1$ decreases $\delta(m)$, increasing the node's likelihood of winning the contention, while $\alpha = 1$ takes into account only the node's distance to the destination.[1] The new equation maintains the offset normalized to S in steps proportional to g, the time gap between a transmitter sensing an available channel and finishing transmission of enough symbols for other nodes to sense a busy channel (Section 2.2).

2.2.2. TSTP greedy forwarding algorithm

To prevent the distortion coefficient from causing messages to be forwarded to an incorrect destination, the TSTP Greedy Forwarding Algorithm (Algorithm 1) ensures that all messages that a node receives and are queued for transmission satisfy the Progress Property: all messages relayed will make positive spatial progress towards the destination. This property can be written as $\forall j \forall i \{m_j i \in Q_i | D_i < D_j\}$, meaning that each message $m_j i$ from node j overheard by node i will be stored in node i's transmission queue Q_i if and only if the distance D_i from node i to the message's destination is smaller than the distance D_j from node j to the same destination.

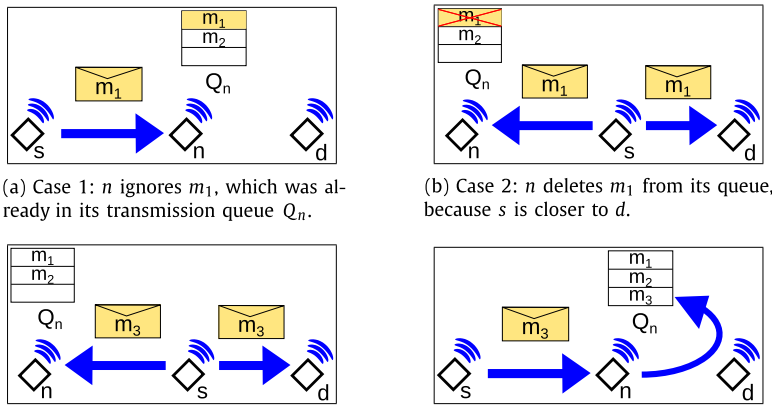
Algorithm 1 handles four possible cases:

1. Case 1 (lines 6, 7): If a copy of the message m is already queued and m is coming from a node farther from the destination, then m is a retransmission attempt and can be ignored (the local copy remains on the queue for later retransmission).
2. Case 2 (lines 8, 9, 10, 11): If a copy of m is already queued and m is coming from a node closer to the destination, then this means that the message has already made positive progress to the destination. The message m is therefore handled as an acknowledgment, causing the local copy e_Q to be removed from the queue Q_i.

[1] A previous version of this equation [21] allowed for α to increase the offset, but it nullified the effect of the spatial factors R, D, D_m. In the current version, if every node uses the same metrics, a node with $\alpha = 1$ will effectively become less likely to win the contention.

Algorithm 1 TSTP greedy forwarding algorithm.

```
1: procedure GreedyForward(m)
2:     queued ← false
3:     for each e_Q ∈ Q_i do
4:         if m.id = e_Q.id then
5:             queued ← true
6:             if Distance(m.LastHop, m.Dst) > Distance(Here, m.Dst) then
7:                 delete m
8:             else
9:                 Q_i.remove(e_Q)
10:                delete m
11:                delete e_Q
12:            end if
13:        end if
14:    end for
15:    if queued = false then
16:        if Distance(Here, m.Dst) > Distance(m.LastHop, m.Dst) then
17:            delete m
18:        else
19:            Q_i.insert(m)
20:        end if
21:    end if
22: end procedure
```



(a) Case 1: n ignores m_1, which was already in its transmission queue Q_n.

(b) Case 2: n deletes m_1 from its queue, because s is closer to d.

(c) Case 3: n ignores m_3, because s is closer to d.

(d) Case 4: n adds m_3 to its queue, because n is closer to d than s is.

Fig. 3. Possible cases for Algorithm 1. In each case, node s sends a message with d as the destination, which is overheard by node n.

3. Case 3 (lines 16, 17): If m is a new message but came from a node closer to the destination, then it means that this node (the receiver) would not make positive progress towards the destination. The message is ignored.
4. Case 4 (lines 18, 19): If m is a new message that came from a node more distant from the destination, then this node can make positive progress towards the destination. It stores the message on its queue and becomes a relay candidate for message m.

Fig. 3 illustrates each of the cases. We prove that the Greedy Forwarding Algorithm ensures the Progress Property as follows:

Theorem 1. *The Greedy Forwarding Algorithm will store in queue Q_i any given incoming messages $m_j i$ from node j overheard by node i if and only if $D_i < D_j$.*

Proof. The proof is by contradiction. Suppose that $\exists i, \exists j, \exists m_j i \{m_j i \in Q_i | D_i \geq D_j\}$ and that every Q is only altered by Algorithm 1. The message $m_j i$ must have been included in Q_i by Case 4 because it is the only case that includes messages in the queue. But Case 4 only includes a given message $m_j i$ in Q_i if $D_i < D_j$, and since by assumption $D_i \geq D_j$, it is not possible that $m_j i$ was included in Q_i by Case 4. Therefore, since Case 4 is the only case that could include $m_j i$ in Q_i, it must be true that $m_j i \notin Q_i$, reaching a contradiction. □

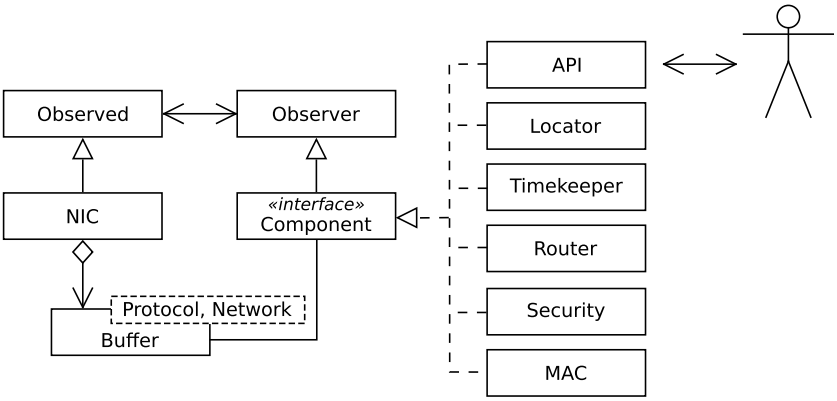


Fig. 4. TSTP architecture overview.

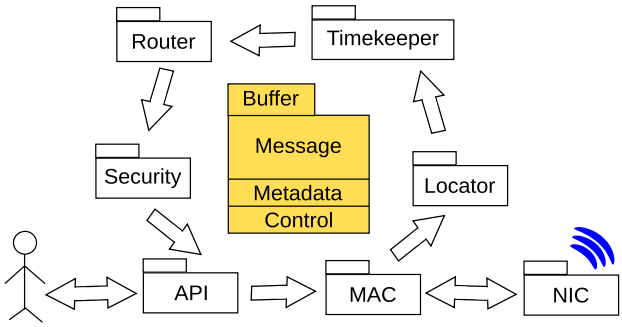


Fig. 5. TSTP component interactions and Buffer life-cycle.

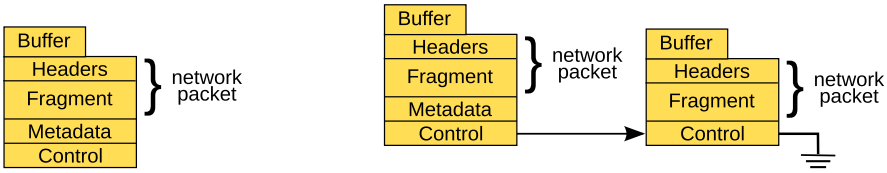
3. Component-based protocol implementation

As explained in the previous section, a cross-layer protocol design can eliminate a large number of control messages and improve decision making within the protocol by pig-tailing control information on ordinary data packets and subsequently sharing that information among the components of the protocol stack. In this section, we discuss the software architecture that was used to implement TSTP's cross-layered design on the *Embedded Parallel Operating System* (EPOS) [13] avoiding a monolithic, tightly-coupled software. We apply template metaprogramming techniques to implement a component-based, event-driven architecture that efficiently moves packets stored in zero-copy buffers enriched with cross-layer metadata across protocol components at the same time as it eliminates unnecessary dependencies.

3.1. Architecture overview

TSTP is implemented as a collaboration of 6 protocol components that interact to implement its cross-layer design: MAC, Router, Timekeeper, Locator, Security, and API. The **Timekeeper** is responsible for precise clock synchronization across the network. It currently implements the *Speculative Precision Time Protocol* (SPTP) [25], but any high-precision protocol could be used. The **Locator** is responsible for keeping spatial coordinates up to date, particularly in nodes devoid of a GPS receiver or for which a static location has not been defined at deployment-time. It currently implements the *Heuristic Cooperative Calibration Positioning System* (HeCoPs) [20]. The **Router** implements the greedy, fully-reactive, geographic routing used by TSTP as it defines and implements the metrics for relay selection described in Section 2.2.1 used by the MAC when forwarding messages [21]. **Security** is responsible for the encryption and authentication of messages and also for the management of cryptographic keys [24]. The **MAC** component manages the communication channel, sending and receiving network packets containing TSTP messages [22]. The **API** handles interactions between the applications and the network through the protocol. Fig. 4 illustrates this architecture.

TSTP messages are stored in zero-copy buffers, which aggregate metadata that is used by the components in the protocol while interacting with each other. Buffers circulate protocol elements in a fixed ordering defined around the six component categories. On receiving, buffers circulate from MAC to API, passing by Locator, Timekeeper, Router, and Security. On sending, they follow the path from API to MAC in the same order, but with a second visit to the MAC. On the first visit to the buffer, MAC populates headers and takes care of the data provided by the application, which is signed and encrypted by Security before returning to MAC for injection into the network. This cycle is depicted in Fig. 5.



(a) Buffer without fragmentation. (b) Buffer with fragmentation.

Fig. 6. EPOS zero-copy buffer optimized by protocol.

3.2. Zero-copy buffers

For the implementation of TSTP, we extended EPOS *Zero-Copy Buffers* [4]. These buffers were first devised aiming at high-throughput, low-latency, low-overhead message exchange between components, but the resource-constrained scenario in which many IoT devices live also requires the memory to be dynamically allocated with parsimony. As in the original version, we transfer the ownership of data structures kept in a ring buffer from one component to another while hiding the unnecessary information. However, differently from the original version, we do not allocate the whole ring buffer at initialization-time. Buffers are allocated from an optimized system's heap on demand. This heap avoids much of the overhead of application-level heaps by assuming allocations and deallocations at system-level are never malicious and thus storing all the needed control information along with the allocated memory chunk.

To receive packets from the network, TSTP allocates a zero-copy buffer passing information about the maximum payload size, the combined size of the non-recurring protocol headers, and the size of the header that must be repeated in case the underlying network requires fragmentation. Allocation takes place considering the MTU of the underlying networks, and the result is a complex object with methods to access any of the elements in the message. Indeed, the buffer is modeled as a parameterized class that takes both the protocol and the underlying network as parameters to perform optimized allocations. On IEEE 802.15.4 networks, TSTP does not require nor allows fragmentation, so the resulting zero-copy buffer structure is the one depicted in Fig. 6a. Other protocol/network arrangements would use the same interface to handle an object like the one in Fig. 6b. The allocated buffer is shared with the network adapter for DMA, and an interrupt is used to signalize that a new packet has been received.[2]

The interrupt trigger by the network adapter is propagated to upper-level protocol components, driving the protocol's state machine and eventually reaching the application whenever a valid message is received. After the application has consumed the buffer, it is released, causing the allocated memory to return to the system's heap. The propagation mechanism is detailed in Section 3.4.

To send packets over the network, TSTP allocates a buffer providing information about all headers and the payload just like it does for receiving. The allocated buffer moves around the protocol components alongside the data given by the application. Each relevant component makes its contributions to the headers, leaving the actual copy of application data to the buffer to the second last component, the MAC. Now with application data, the buffer goes to the security component for encryption and signature. It finally returns to the MAC for injection in the network.

3.3. Metadata

The zero-copy buffer parameterized class described above takes both the protocol and the target network as parameters. In this way, buffers can be extended with additional, protocol-specific *metadata*. Instead of sharing their data structures to exchange collected knowledge about the network, components use the metadata in the buffer for interaction with each other. The metadata is usually not transmitted over the network along with messages. If they do, they go pig-tailed to application data in the payload. When a message is received at a node, each component that sees the corresponding buffer populates and adjusts specific portions of the metadata contained in that buffer. Similarly, messages coming from the application are encapsulated in a buffer that traverses the protocol from component to component. Both directions follow an ordering as shown in Fig. 5.

The main metadata used in TSTP are:

RSSI: a *Received Signal Strength Indicator* or similar signal indicator that is captured on packet arrival and used by the Locator while keeping track of the node's position in space.

SFD timestamp: a timestamp captured as soon as the delimiter of the start of a new frame is observed in the network. It is used by the Timekeeper to synchronize the node's clock with the sender of that message.

[2] EPOS zero-copy buffers support network adapters that can handle multiple packets asynchronously. In this case, the whole ring buffer is shared with the adapter, which has full access to the memory bus and utilizes an atomic, per-buffer lock shared with the main processor to synchronize concurrent accesses. This medium is the case for most Ethernet adapters on Intel and many ARM platforms, but it is not used in TSTP for IEEE 802.15.4 to preserve as much memory as possible free for the application.

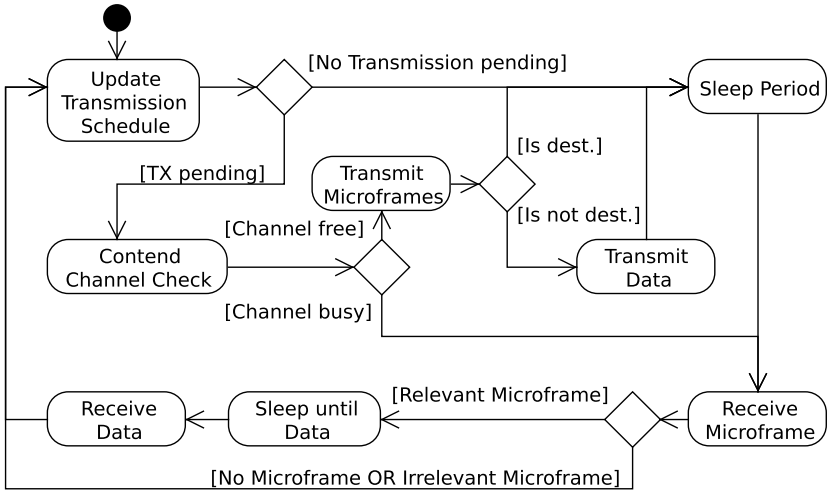


Fig. 7. TSTP MAC's activity diagram.

Offset: The time the node sleeps in the `Contend` state of the MAC. Calculated by the Router.

Destined to me: Whether this message is destined to this node. Set by the Locator.

Direction: Whether this message goes towards the gateway. Taken from the `All Listen` microframe field.

Distance: The distance from this node to the message's destination. Calculated by the Locator.

Sender distance: The distance from the sender of this message to its destination. Taken from the `Distance` microframe field.

Trusted: Security sets these bits if it can verify the message's integrity and authenticity.

Relevant: Any component may set this bit when processing a microframe to signal to the MAC that it should receive the corresponding message. For instance, the Timekeeper might need to get a message to resynchronize the clock, or the Locator might need to reinforce its confidence about its location.

3.4. Event propagation

Events are propagated in the protocol in two different flows: from application to the network, and from network to application. On the first path, from application to the network, events are implicitly propagated as components are invited to visit a buffer in a predefined order through the invocation of specific methods that collaboratively marshal a valid network packet. On the counterflow, from the network up to the application, events are propagated using the Observer design pattern [9]. When the network adapter finishes copying a packet into a previously allocated buffer and triggers an interrupt, that interrupt is handled by the device driver (called *Network Adapter Interface* (NIC)) and converted into a *notification* of an observed object (the NIC). TSTP protocol components are therefore observers of zero-copy buffers.

3.5. MAC state machine

As explained in Section 2.2, the MAC component interfaces the network to other TSTP components implementing a low duty cycle, receiver-based, packetized-preamble-sampling scheme that supports arbitrary metrics for relay selection. Fig. 7 shows the activity diagram of TSTP MAC. The `Update Transmission Schedule` activity maintains the transmission queue Q_i, checking for pending buffer transmission. During `Contend Channel Check`, the node sleeps for a time defined by Equation (4) and checks the channel. If free, the node proceeds to `Transmit Microframes` for a full period S, with the strictly defined timing explained in Section 2.2. If the node is the final destination for this message, the microframes just sent serve as an acknowledgment, and no data transmission is required (it will make previous nodes drop the message from their queues as a consequence of Algorithm 1). Otherwise, it proceeds to `Transmit Data`, a simple transmission of the queued message. In `Sleep Period`, nodes sleep for the predefined period S, and during `Receive Microframe` they listen to the channel for a time of $2T_s + T_u$ (Section 2.2). If a microframe is received, the node decides whether or not the message is relevant. If the message is relevant, the node sleeps until the time the message arrives (the time is derived from the `Count` field in the microframe, the known time between microframes T_u, and the known time to transmit a single microframe). The `Receive Data` activity simply listens until a data message arrives, with a timeout for the case of interference. When the channel is idle, the MAC will alternate between `Update Transmission Schedule`,

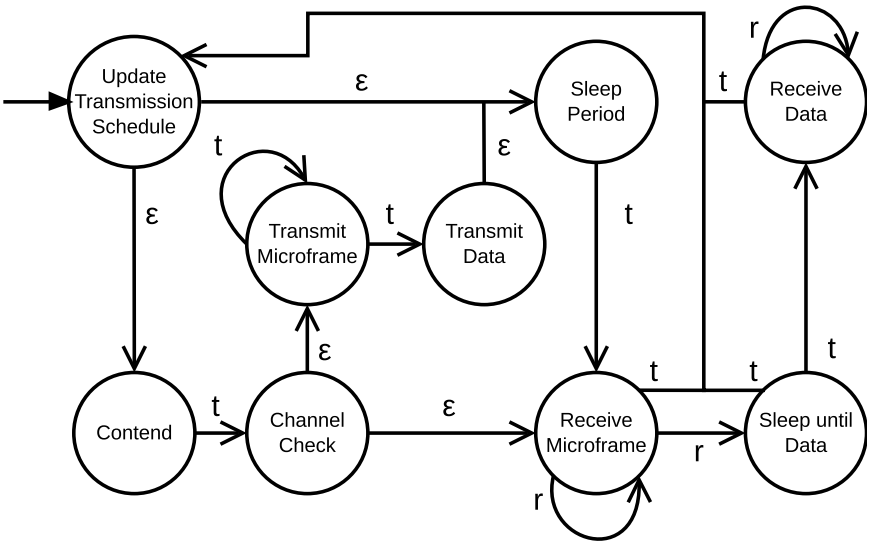


Fig. 8. TSTP MAC implementation's automaton. t transitions are triggered by timer interrupts, r denotes radio interrupts, and ϵ represents a transition that happens without a hardware interrupt (e.g. a function call at the end of the previous state's function).

`Sleep Period`, and `Receive Microframe`. The idle listening duty cycle thus arises from the adjustment of the period S spent in `Sleep Period` as a function of the (constant) time spent in `Receive Microframe`, as shown in Section 2.2.

Because TSTP MAC deals with very precise and constrained timing, its states are implemented as interrupt handlers for the NIC and a dedicated timer. Fig. 8 shows a Nondeterministic Finite Automaton (NFA) for the MAC, referencing the software functions that implement each activity shown in Fig. 7 and the hardware events that cause state transitions. This NFA was also used to check the behavior of the MAC state machine during development. We instrumented the code to indicate (by printing a character on the screen) whenever a state is entered or a relevant hardware interrupt happened and collected these execution traces. Then, we rebuilt the same NFA as a separate program which parses the traces and checks whether any state violation is found. This process was particularly important because Interrupt Service Routines (ISR) can be preempted in EPOS. ISR preemption becomes more likely to occur as the MAC deals with events that happen close to each other in time. With these traces and checks, we have evidence that there is no unintended preemption of handlers that lead to inconsistent state transitions.

4. Protocol evaluation

To evaluate the design of TSTP described in this paper and therefore corroborate the claims made about the advantages brought by its cross-layered organization and its component-based implementation, we have devised two sets of experiments: the first focuses on the cross-layered design, while the second targets the component-based implementation. Both are detailed next.

4.1. Communication performance

We compare TSTP with other traditional protocols in the scene of embedded IoT devices through a set of OMNET++ [18] (version 4.6) simulations using the Castalia [2] (version 3.3) framework. Before comparing TSTP with other protocols, we assessed the quality of our simulation models by comparing the simulation results with the ones obtained in-field on our Solar Smart Building, which is automated using EPOS and TSTP running on EPOSMoteIII hardware. Indeed, the source code used for the simulations is basically the same used in the real deployment, but the comparison between the real scenario and the simulated model allowed us to adjust the model's parameters in order to obtain realistic results.

Next, we present two metrics over the distortion coefficient that were used in the simulations along with the original, undistorted, geographic distance: one metric is the proximity to message expiration (Equation (5)), and the other is the routing effort metric (Equation (6)). The two distortion coefficients described next are applied to the offset formula (Equation (4)) to influence the decision of which node will forward a message, as explained in Section 2.2.1.

4.1.1. Expiry metric

The Expiry metric defines a distortion coefficient influenced by how much time the message has left to reach its destination. Messages that are close to expiring shrink space, reducing the node's offset $\delta(m)$, and providing it a better chance to win the contention and forward that message quickly. Equation (5) defines the distortion coefficient for the Expiry metric.

Table 1
Simulation deployment parameters.

Nodes	100
Field size	200 m × 200 m
Deployment	10 × 10 regular grid
Radio Sensitivity	−85 dBm
Transmission Power	0 dBm
Propagation Model	Free Space Model

$$\alpha = \frac{E_m - St_m}{E_m} \tag{5}$$

where E_m is the timestamp in which message m expires and St_m is the time already spent since the message's generation time. Both E_m and St_m are taken from the message's header (Fig. 1).

4.1.2. Effort metric

The Effort metric distorts space based on how much the node has cooperated with the network doing message forwarding in the past. The more messages the node has forwarded, the more space is dilated, and the less likely that node becomes to forward again. This metric seeks a balance on the network for neighbors on the same hop from the destination to better alternate the effort of forwarding messages. Equation (6) defines the distortion coefficient for the effort metric.

$$\alpha = \frac{F_n}{Ts_n} \tag{6}$$

where F_n is the number of queued messages from neighbors (according to Algorithm 1, explained in Subsection 2.2.2) that node n has relayed so far. Ts_n is the total number of messages that the node has transmitted in the past, including its own generated messages. The effort metric takes information from the node's history, rather than from information related to the message being forwarded.

4.1.3. Simulation parameters

The simulated network is composed of 100 nodes deployed over a 200 m by 200 m field on a 10 by 10 grid formation. Table 1 summarizes the simulation deployment parameters. We compare the following implementations:

TSTP no distortion: TSTP without any distortion metrics.

TSTP Effort: TSTP with the Effort metric (Equation (6)) as a distortion coefficient.

TSTP Expiry: TSTP using the Expiry metric (Equation (5)) as a distortion coefficient.

AODV and AODVT: two different third-party implementations of the AODV protocol for the Castalia framework.

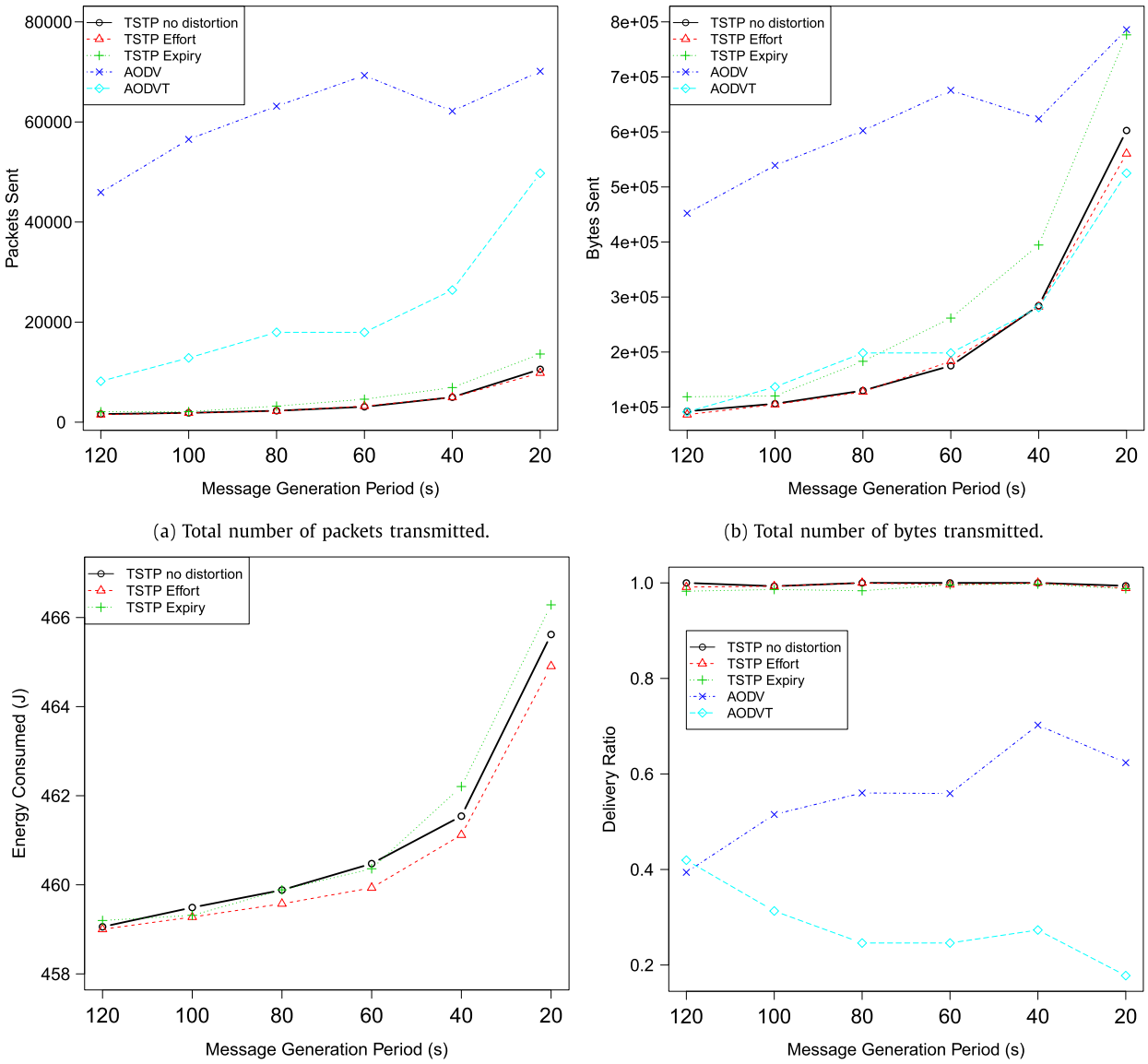
TSTP implementations run at 20% duty cycle with an expiry of 1 s (every message that takes more than one second to get to its destination is dropped). AODV and AODVT run on top of an IEEE 802.15.4 MAC layer with the basic CSMA mechanism. It does not implement duty cycling.

Every node in the network generates one message every s seconds, which is routed to the sink at the upper-left corner of the grid. The period s varies from 20 to 120 along the x axes in Fig. 9. To evaluate the impact of TSTP's pig-tailing of spatial and temporal information on messages instead of using explicit message exchanges for synchronization, for the AODV implementations we also run application-level PTP and HeCoPs with the same period s to simulate a non-cross-layer spatial and temporal synchronization.

The application-level PTP synchronizes clocks by exchanging three messages: `sync`, `delay request` and `delay response`. A master node (the sink) sends a `sync` as a mac-level broadcast every period s, which is responded to with a `delay request` by all its neighbors. The master then sends a unicast `delay response` to each. Each newly synchronized node then becomes a master and restarts the process for all its neighbors that did not receive the previous `sync` messages, and so on until all nodes are synchronized. The application-level HeCoPs simply generates a one-hop broadcast every s seconds at every node, through which nodes share their spatial information.

Fig. 9 shows the simulation results comparing the implementations of TSTP and AODV, taking into account the number of packets and bytes sent during the active period of each protocol (20% for TSTP, 100% for AODV). Since OMNET++ only advances the simulated time in an event-by-event basis and we are not measuring the simulation running time, the code in the application and radio components that collects the results shown do not impact in any way on the results themselves.

Because the MAC used with AODV does not manage the radio energy consumption, this metric is not comparable with these protocols. However, the data in Fig. 9c shows that TSTP running at 20% idle listening duty cycle indeed consumes around 28% as much energy as a protocol that leaves the radio always on (AODV and AODVT consumed from 1665 J to 1666 J each). In Figs. 9a, 9b, we can see that the protocol overhead caused by TSTP's pig-tailing of geographic information on application data messages is far smaller than the total overhead caused by the routing protocol and the explicit exchange of temporal and spatial synchronization messages. Although Fig. 9b indicates that AODVT causes a lower overhead in some



(a) Total number of packets transmitted.

(b) Total number of bytes transmitted.

(c) Consumed energy.

(d) Delivery Ratio.

Fig. 9. Simulation results for various protocols. AODV and AODVT are not shown in (c), as the gap would make the small increase in TSTP consumption indiscernible. Power consumption for both implementations is close to constant, varying from 1665 J to 1666 J.

scenarios, Fig. 9d reveals that this low overhead occurs because as much as 80% of the application's data packets were dropped along the route. TSTP is successfully able to save energy, messages, bytes, and keep the delivery ratio close to 100%.

4.2. Software overhead

As mentioned earlier, TSTP's component-based implementation and a programming language compatibility allowed it to be ported from EPOS to Castalia without much effort: mostly rewriting the NIC component to handle Castalia's Radio framework, and the TSTP API's notification method to interact with Castalia's application layer.[3] Table 2 shows the resulting compiled code size for each protocol. It shows EPOS' and Castalia's implementations of TSTP, two third-party implementations of the AODV protocol, and BypassRouting, a very simple network layer in Castalia that simply forwards messages

[3] A few other technical changes were necessary, such as the removal of static methods and class attributes, which in Castalia are shared across every node in the simulated network.

Table 2
Code size for each implementation.

Implementation	Code size (bytes)
BypassRouting	410
Castalia TSTP	12346
EPOS TSTP	12554
AODV	29565
AODVT	35389

Table 3
Assessment of time overhead for buffer management.

Buffers allocated	260548
Avg. buffer allocation time	2.31 µs
Received buffers processed	278800
Avg. buffer processing time	108.57 µs
Avg. notify time	12.60 µs
Notify time/processing time	11.60%

directly between the MAC and application layers. Both TSTP implementations are very close in size because of their similarity and are significantly lighter than both the AODV implementations. This size does not include the statistics collection methods.

Castalia code (BypassRouting, Castalia TSTP, AODV, AODVT) runs on a simulator and was compiled for an Intel Core i5-2320 CPU, g++ version 4.8.5-2ubuntu1 14.04.1, on a x86_64 Ubuntu 14.04 Operating System. EPOS code (EPOS TSTP) runs on a real IoT device (EPOSMoteIII) and was cross-compiled with g++ version 4.4.4 for the ARM Cortex-M3 processor.

4.2.1. Buffer management

As explained in Section 3.2, TSTP uses a relatively simple version of EPOS' zero-copy buffer and a system heap to allocate buffers efficiently. Apart from TSTP metadata and the IEEE 802.15.4 PDU, the buffer uses a one-byte lock, a four-byte owner pointer, and a four-byte size counter as control information, summing up to 9 bytes of overhead.

We measured the time overhead for allocation of buffers on two active EPOSMoteIII IoT devices running TSTP, one acting as a sink and the other as a sensor node. The two nodes allocated a total of 260548 buffers, taking on average 2.31 µs for each allocation. EPOSMoteIII is equipped with Texas Instrument's CC2538 System-on-Chip, with an ARM Cortex-M3 processor at 32 MHz. For the experiments in this section, we disabled hardware interrupts during the execution of the code parts being assessed.

The mechanism for propagating received buffers throughout the components explained in Section 3.4 was evaluated on the same EPOSMoteIII devices. We record the time a buffer enters and leaves each component to find the propagation overhead in relation to the total processing time. For 278800 measurements, the average time for each buffer to leave the MAC, be processed by the other components, and return to the MAC (as depicted in Fig. 5) was 108.57 µs. From this time, on average 12.60 µs were spent between the end of a component's update method and the start of the next, accounting for the overhead of the notification mechanism itself, which represented 11.60% of the total buffer processing time. Table 3 summarizes these measurements.

Since every component has access to the full packet data and metadata, it is not easy to guarantee that components will not change pieces of information that other components may use in another way. There is no integrity control in our current implementation of the zero-copy buffer. Besides added overhead, it is not straightforward to enforce integrity, because one of our implementation's design goals is to keep the components free to use and modify any information, as well as make it easier to try new, previously unexpected subcomponents. The only form of control currently implemented is the well-defined order in which the components' methods for handling buffers are called.

5. Related work

Several protocols have been designed in a cross-layered style to improve different aspects of their architecture. The main motivations for a cross-layer design are to reduce redundancy of functionalities over the stack and to ease the sharing of information across components. The way that components are connected through the stack is directly related to the application's requirements. Timing, energy consumption, delivery ratio, and end-to-end delay are common requirements for WSN. The protocols are designed to reduce the power consumption of the network, increase delivery ratio, decrease the end-to-end delay, and meet timing constraints. TSTP goes further, providing security and space-time synchronization. In this section, we present a few cross-layer proposals in the literature.

Fu et al. [8] analyze several cross-layer designs for wireless networks and identify advantages, goals, a classification, and inherent disadvantages of cross-layer wireless architectures in general. According to the authors' classification, TSTP can be seen as a *Manager* method regarding information sharing between components on a same node. The buffer cycle acts a vertical data exchange plane, which removes explicit interaction between components (as in *Non-manager* methods), making the components less tightly coupled to each other. Regarding data sharing between nodes, TSTP uses the *Distributed*

method, which does not use any hierarchy or central node for information sharing, thus improving network resiliency in principle. TSTP is affected by the main challenges identified by the authors. We have shown that we share cross-layer data between nodes with TSTP message headers, and our experiments show that the inter-node *Overhead caused by cross-layer signaling* does not make TSTP less efficient than other non-cross-layer protocols. We have also shown that the buffer management mechanism is light in memory and time overhead, and thus the intra-node *Overhead caused by cross-layer signaling* is modest. The *Destruction of the layered architecture* disadvantage states that cross-layer designs lose the high modularity and standardization provided by black-box designs, and in general any modification to any layer must have the design and implementation of the whole stack in mind. While this is true in some level in TSTP, it is mitigated by the vertical data-sharing plane, which grants some level of independence to each component. Problems of *Coexistence* and *The lack of the universal cross-layer design* arise because each cross-layer design is usually proposed with a specific application or optimization in mind, and different applications in a same system might have very different requirements. This is less of a problem in the embedded, application-oriented context of EPOS and TSTP, where by design the whole system is tailored to a single application [7].

TSTP MAC [22], presented in this work, is an adaptation and implementation of Receiver-Based MAC (RB-MAC) [1] under the TSTP context. RB-MAC defines the cooperative, distance-based geographic forwarder selection and passive acknowledgment mechanisms used initially in TSTP MAC. The authors of RB-MAC analyze the protocol regarding energy consumption and latency, comparing it to 1-hopMAC, where senders define a specific receiver at each hop. They show that RB-MAC is in general more energy efficient and causes less latency because its anycast nature makes it much more resilient to lossy links [1]. Furthermore, not defining a receiver allows for dynamic and diverse contention metrics, although the authors do not investigate their impact as we do in previous work [21]. In previous work [28], we implemented both RB-MAC and B-MAC under a Configurable MAC framework in the Embedded Parallel Operating System (EPOS), and showed that RB-MAC far outperforms B-MAC for varying channel conditions. In another work [23], we also analyzed the impact of the latency caused by RB-MAC in TSTP.

TSTP MAC is mainly concerned with dealing with a lossy and dynamic environment, optimizing the number of message exchanges, and saving energy, but one of its main drawbacks is in latency. The CMAC [15] protocol uses a similar routing idea as TSTP, and it solves the latency problem. Messages can be sent in unicast or anycast modes. In anycast, when a receiver closer to the destination detects a microframe (called RTS in CMAC), it backs off for a period that might be influenced by an arbitrary routing metric. Then, if the channel is free, it sends back a CTS, and the sender *immediately* stops the preamble and transmits the data packet. The main difference from TSTP MAC in this aspect is that cutting the preamble makes many possible forwarding candidates not have a chance to detect the transmission and passively benefit from information in the data packet if needed, as well as provide transmission redundancy. If the receiver that replied with a CTS is in a good enough region, the sender locks in and will start unicasting to this node in future transmissions until one of them fails; if this happens, the sender returns to anycast mode. During unicast mode, nodes may synchronize and coordinate transmissions to optimize latency, but redundancy and overhearing is also lost, making the network less robust and reactive to lossy links.

Glossy [6] is an efficient one-to-all communication architecture for Wireless Sensor Networks that presents low latency with a different approach. It is a cross-layer solution comprising routing, MAC, time synchronization, and network API. Similar to TSTP's Speculative Precision Time Protocol [25], Glossy exploits control over the MAC layer to synchronize clocks over the network with low cost and high accuracy as a consequence of the well-defined communication timings. Glossy exploits constructive interference on IEEE 802.15.4 networks to flood packets with high energy and time efficiency, but achieving constructive interference brings very strict timing requirements. Thus, the authors detail the techniques used to reach a highly time-deterministic implementation. LWB [5] extends Glossy to the context of data-gathering WSN, with many nodes initiating data transmissions. Glossy and LWB present many, but not all of TSTP's features: high accuracy time synchronization; reactive, fully distributed, topology-agnostic routing; and a low power MAC with high redundancy (in case one node fails, its neighbors will continue the transmission). However, multiple different Glossy floods happening at once would greatly interfere with each other, and so LWB uses a global schedule to ensure that no two floods overlap in time. This global scheduling requires knowledge about all the devices on a network, weakening the advantages of an ad-hoc routing. Nevertheless, LWB and Glossy latency and energy consumption results are impressive, and as future work we plan to study improvements to TSTP MAC, possibly using similar techniques.

6. Conclusion

In this work, we presented the cross-layer design of the Trustful Space-Time Protocol (TSTP), which ranges from the application to the MAC layer, providing geographic routing, duty cycling, time and space synchronization, security, and an API. We went into detail on the interactions between MAC and router, which encompass many of the challenges that arise in the cross-layer design.

We showed how we use an *event propagation* mechanism to share *metadata*-enriched *zero-copy buffers* across components with low overhead, allowing each component to be independent of the others and achieving modularity.

The protocol's cross-layer design was evaluated in terms of communication overhead by a set of OMNeT++ simulations, which showed that TSTP's pig-tailing of geo-synchronization information is indeed able to save bandwidth when compared

to explicit exchange of synchronization messages, while still keeping low energy consumption and a high delivery ratio of application data.

We also measured time and memory overhead for the main buffer management mechanisms on a real IoT platform, showing that we achieved a light implementation of TSTP's full cross-layer design.

Although there is room for improvement in many aspects of several components, we have shown that it is possible to realize a tightly coupled cross-layered design encompassing the full network functionality with a light, efficient, and loosely coupled implementation. The freedom this infrastructure provides to each component allows for experimentation and improvement on each one independently, covering each limitation incrementally and contributing to a highly optimized IoT solution, one step at a time.

References

[1] Mohammad R. Akhavan, Thomas Watteyne, A. Hamid Aghvami, Enhancing the performance of RPL using a receiver-based MAC protocol in lossy WSNs, in: IEEE ICT, Ayia, Napa, Cyprus, May 2011, pp. 191–194.

[2] Athanassios Boulis, Castalia A simulator for wireless sensor networks and body area networks, 2017.

[3] Colin Dixon, Ratul Mahajan, Sharad Agarwal, A.J. Brush, Bongshin Lee, Stefan Saroiu, Paramvir Bahl, An operating system for the home, in: Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation, NSDI'12, USENIX Association, Berkeley, CA, USA, 2012, pp. 25–25.

[4] Thiago Robert dos Santos, Antônio A. Fröhlich, A customizable component for low-level communication software, in: 19th Annual Symposium on High Performance Computing Systems and Applications, Guelph, Canada, May 2005, pp. 58–64.

[5] F. Ferrari, M. Zimmerling, L. Thiele, L. Mottola, The bus goes wireless: routing-free data collection with QoS guarantees in sensor networks, in: 2012 IEEE International Conference on Pervasive Computing and Communications Workshops, March 2012, pp. 26–31.

[6] F. Ferrari, M. Zimmerling, L. Thiele, O. Saukh, Efficient network flooding and time synchronization with glossy, in: Proceedings of the 10th ACM/IEEE International Conference on Information Processing in Sensor Networks, April 2011, pp. 73–84.

[7] Antônio A. Fröhlich, Application-Oriented Operating Systems, GMD–Forschungszentrum Informationstechnik, Sankt Augustin, 2001.

[8] Bo Fu, Yang Xiao, Hongmei Deng, Hui Zeng, A survey of cross-layer designs in wireless networks, IEEE Commun. Surv. Tutor. 16 (1) (2014) 110–126.

[9] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison–Wesley, 1995.

[10] J. Granjal, E. Monteiro, J. Sá Silva, Security for the internet of things: a survey of existing protocols and open research issues, IEEE Commun. Surv. Tutor. 17 (3) (2015) 1294–1312.

[11] P. Huang, L. Xiao, S. Soltani, M.W. Mutka, N. Xi, The evolution of MAC protocols in wireless sensor networks: a survey, IEEE Commun. Surv. Tutor. 15 (1) (2013) 101–120.

[12] A. Hulbert, T. Kunicki, J.N. Hughes, A.D. Fox, C.N. Eichelberger, An experimental study of big spatial data systems, in: IEEE International Conference on Big Data, Big Data, Dec. 2016, pp. 2664–2671.

[13] Software/Hardware Integration Lab. EPOS – Embedded Parallel Operating System, 2017.

[14] Philip Levis, Neil Patel, David Culler, Scott Shenker, Trickle: a self-regulating algorithm for code propagation and maintenance in wireless sensor networks.

[15] S. Liu, K.W. Fan, P. Sinha, CMAC: an energy efficient MAC layer protocol using convergent packet forwarding for wireless sensor networks, in: 2007 4th Annual IEEE Communications Society Conference on Sensor, Mesh and Ad Hoc Communications and Networks, June 2007, pp. 11–20.

[16] Lucas D.P. Mendes, Joel J.P.C. Rodrigues, A survey on cross-layer solutions for wireless sensor networks, J. Netw. Comput. Appl. 34 (2) (2011) 523–534.

[17] Alexandre M. Okazaki, Antônio A. Fröhlich, ADHOP: an energy aware routing algorithm for mobile wireless sensor networks, in: IEEE SENSORS, Taipei, Taiwan, October 2012.

[18] OpenSim, OMNeT++ – Objective Modular Network Testbed in C++, 2017.

[19] Mallanagouda Patil, Rajashekhar C. Biradar, A survey on routing protocols in wireless sensor networks, in: 2012 18th IEEE International Conference on Networks, ICON, IEEE, 2012, pp. 86–91.

[20] Ricardo Reghelin, Antônio A. Fröhlich, A decentralized location system for sensor networks using cooperative calibration and heuristics, in: 9th ACM/IEEE International Symposium on Modeling, Analysis and Simulation of Wireless and Mobile Systems, Torremolinos, Malaga, Spain, October 2006, pp. 139–146.

[21] Davi Resner, Gustavo Medeiros de Araujo, Antônio A. Fröhlich, On the impact of dynamic routing metrics on a geographic protocol for WSNs, in: 2016 VI Brazilian Symposium on Computing Systems Engineering, SBESC, João Pessoa, Brazil, November 2016, pp. 109–115.

[22] Davi Resner, Antonio A. Frohlich, TSTP MAC: a foundation for the trustful space-time protocol, in: 2016 IEEE Intl Conference on Computational Science and Engineering (CSE) and IEEE Intl Conference on Embedded and Ubiquitous Computing (EUC) and 15th Intl Symposium on Distributed Computing and Applications for Business Engineering (DCABES), EUC, Paris, France, August 2016, pp. 40–47.

[23] Davi Resner, Antônio A. Fröhlich, Design rationale of a cross-layer, trustful space-time protocol for wireless sensor networks, in: 20th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA, Luxembourg, September 2015, pp. 1–8.

[24] Davi Resner, Antônio A. Fröhlich, Key establishment and trustful communication for the Internet of things, in: 4th International Conference on Sensor Networks, SENSORNETS 2015, Angers, France, February 2015, pp. 197–206.

[25] Davi Resner, Antônio A. Fröhlich, Lucas F. Wanner, Speculative precision time protocol: submicrosecond clock synchronization for the IoT, in: 21th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA, Berlin, Germany, September 2016, pp. 1–8.

[26] Thomas Schmid, Prabal Dutta, Mani B. Srivastava, High-resolution, low-power time synchronization an oxymoron no more, in: Proceedings of the 9th ACM/IEEE International Conference on Information Processing in Sensor Networks, IPSN '10, ACM, New York, NY, USA, 2010, pp. 151–161.

[27] Vineet Srivastava, Mehul Motani, Cross-layer design: a survey and the road ahead, IEEE Commun. Mag. 43 (12) (Dec 2005) 112–119.

[28] Rodrigo V. Steiner, Mohammad R. Akhavan, Antônio A. Fröhlich, A. Hamid Aghvami, Performance evaluation of receiver based MAC using configurable framework in WSNs, in: IEEE Wireless Communications and Networking Conference, WCNC, Shanghai, China, April 2013, pp. 884–889.